

CHỦ ĐỀ: SỬ DỤNG TRÍ TUỆ NHÂN TẠO CÓ TRÁCH NHIỆM VÀ ĐẠO ĐỨC TRONG HỌC TẬP VÀ NGHIÊN CỨU KHOA HỌC

- Họ và tên sinh viên: Phan Nhật Quân
- Mã số sinh viên: 25020769
- Lớp/Khóa: Khóa QH-202X-I/CQ, Ngành Kỹ thuật máy tính
- Trường: Đại học Công nghệ - Đại học Quốc gia Hà Nội (UET - VNU)

I. ĐẶT VẤN ĐỀ

- Sự bùng nổ của các mô hình ngôn ngữ lớn (LLMs) và trí tuệ nhân tạo tạo sinh (Generative AI) như ChatGPT, Gemini, hay Claude đã và đang tái định hình toàn bộ nền giáo dục đại học, đặc biệt là trong khối ngành Công nghệ thông tin (IT). Đối với một sinh viên ngành Công nghệ thông tin tại Trường Đại học Công nghệ (UET - VNU), AI không chỉ là một công cụ tìm kiếm thông tin thông thường mà đã trở thành một "trợ lý lập trình" đắc lực, hỗ trợ từ việc giải thích thuật toán, sửa lỗi mã nguồn (debug) cho đến tối ưu hóa kiến trúc hệ thống.
- Tuy nhiên, việc sử dụng AI quá dễ dàng cũng đặt ra những thách thức lớn về đạo đức học thuật, nguy cơ đạo văn, và sự thui chột tư duy độc lập nếu sinh viên lạm dụng một cách thụ động. Bài báo cáo này được thực hiện nhằm mục đích nghiên cứu chính sách sử dụng AI trong môi trường đại học, thực nghiệm quy trình tương tác với AI có kiểm duyệt để hoàn thành một nhiệm vụ học thuật cụ thể, từ đó đúc kết bộ nguyên tắc ứng xử có trách nhiệm với công nghệ.

II. MÔ TẢ QUÁ TRÌNH THỰC HIỆN NHIỆM VỤ HỌC TẬP VỚI SỰ HỖ TRỢ CỦA AI

Nhiệm vụ học tập được lựa chọn để thực hiện là: **Tổng hợp tài liệu chuyên ngành về ưu, nhược điểm của hai mẫu thiết kế (Design Patterns) Singleton và Observer trong môi trường lập trình đa luồng (Multi-threaded)**. Quá trình tương tác với công cụ AI được thực hiện qua hai lượt Prompt có chủ đích nhằm đạt được kết quả chuyên sâu nhất.

1. Lượt tương tác thứ nhất: Khảo sát lý thuyết tổng quan

- Prompt 1 đã sử dụng:** > Tôi là sinh viên ngành IT. Tôi đang làm một bài tổng hợp tài liệu về các mẫu thiết kế (Design Patterns). Hãy tóm tắt cho tôi định nghĩa, ưu điểm và nhược điểm của hai Pattern: **Singleton** và **Observer** khi áp dụng trong hệ thống đa luồng (Multi-threaded). Hãy viết ngắn gọn dưới dạng danh sách (bullet points)
- Kết quả đầu ra của AI:** AI đã liệt kê đầy đủ định nghĩa cơ bản của hai mẫu thiết kế. Đối với hệ thống đa luồng, AI chỉ ra được nguy cơ tranh chấp tài nguyên (Race Condition) của Singleton và lỗi bất đồng bộ dữ liệu (ConcurrentModificationException), nguy cơ khóa chết (Deadlock) của Observer.

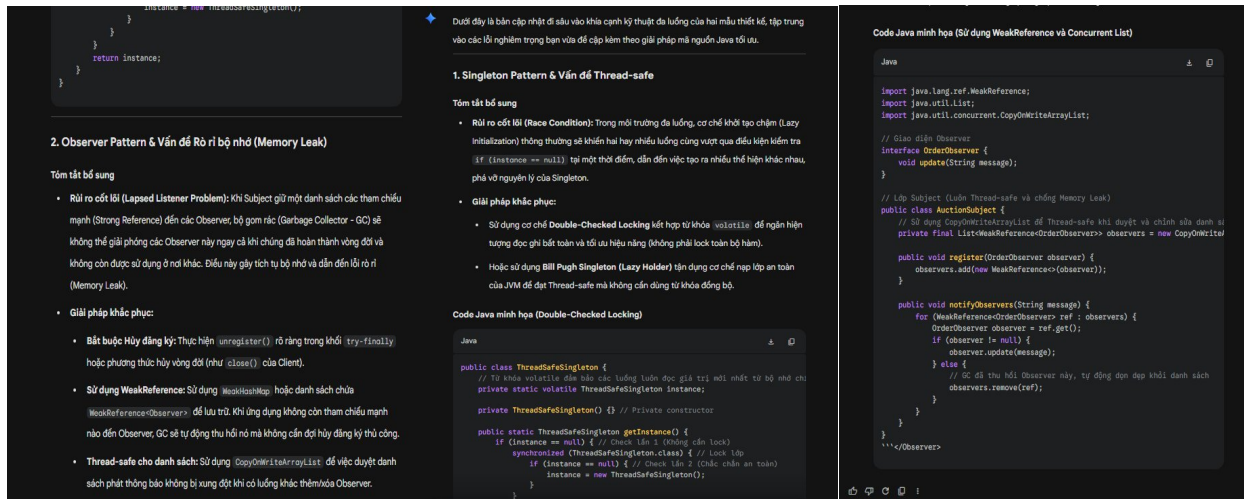
1. Singleton Pattern Đảm bảo một lớp chỉ có duy nhất một thể hiện (instance) và cung cấp một điểm truy cập toàn cục cho thể hiện đó. Ưu điểm trong môi trường đa luồng: Quản lý tài nguyên tập trung: Rất hiệu quả khi dùng để quản lý các tài nguyên dùng chung trong hệ thống đa luồng như Thread Pool, Connection Pool, hoặc cấu hình (Configuration). Tránh xung đột cấu hình: Đảm bảo các luồng khác nhau không tạo ra nhiều trạng thái mâu thuẫn của cùng một tài nguyên hệ thống. Nhược điểm & Thách thức: Nguy cơ tranh chấp tài nguyên (Race Condition): Nếu không được đồng bộ hóa đúng cách, nhiều luồng có thể khởi tạo đồng thời dẫn đến việc tạo ra nhiều thể hiện khác nhau. Giảm hiệu năng do đồng bộ hóa (Overhead): Việc sử dụng các cơ chế khóa (như <code>synchronized</code> trong Java) để bảo vệ quá trình khởi tạo (ví dụ: áp dụng <code>Double-Checked Locking</code>) hoặc bảo vệ phương thức truy cập sẽ làm giảm tốc độ xử lý của các luồng.	2. Observer Pattern Định nghĩa mối quan hệ một-nhiều giữa các đối tượng, sao cho khi một đối tượng thay đổi trạng thái, tất cả các đối tượng phụ thuộc (Observers) của nó sẽ tự động được thông báo và cập nhật. Ưu điểm trong môi trường đa luồng: Giao tiếp bất đồng bộ (Asynchronous): Đối tượng chính (Subject) có thể đẩy thông báo cho các luồng xử lý của Observer mà không cần đợi chúng xử lý xong, giúp hệ thống không bị nghẽn (Non-blocking). Khả năng mở rộng (Scalability): Dễ dàng phân phối việc xử lý dữ liệu từ một nguồn đến nhiều luồng Observer độc lập chạy song song. Nhược điểm & Thách thức: Lỗi bất đồng bộ dữ liệu (Thread-Safety): Danh sách quản lý các Observer có thể bị lỗi (như <code>ConcurrentModificationException</code>) nếu một luồng đang duyệt qua danh sách để thông báo, trong khi một luồng khác lại thêm/bớt Observer. Nguy cơ Khóa chết (Deadlock): Nếu phương thức cập nhật (<code>update()</code>) của Observer lại gọi ngược hoặc đồng bộ hóa (lock) lẫn nhau với phương thức của Subject, hệ thống rất dễ rơi vào trạng thái đóng băng hoàn toàn. Để tối ưu hóa hai mẫu này trong hệ thống đa luồng (như hệ thống mạng, đấu giá, chat...), bạn nên cân nhắc sử dụng các cấu trúc dữ liệu thread-safe sẵn có (như <code>CopyOnWriteArrayList</code> cho Observer) hoặc cơ chế <code>Initialization-on-demand holder</code> cho Singleton để tối ưu hiệu năng.
---	--

2. Lướt tương tác thứ hai: Đánh giá phản biện và đào sâu kỹ thuật

- Đánh giá của sinh viên về kết quả 1:** Nội dung đầu tiên của AI tuy đúng về mặt lý thuyết nhưng còn quá chung chung, thiếu các giải pháp lập trình thực tế bằng mã nguồn – điều cốt lõi mà một kỹ sư IT cần có. Do đó, tôi tiến hành đưa ra câu lệnh thứ hai nhằm kiểm tra khả năng giải quyết vấn đề sâu hơn của AI.
- Prompt 2 đã sử dụng:**

"Phần tóm tắt lý thuyết rất tốt. Để hoàn thiện tài liệu, hãy cung cấp cho tôi đoạn code Java ngắn gọn minh họa cách triển khai tốt nhất cho 2 mẫu này trong môi trường đa luồng: 1. Với Singleton: Dùng cơ chế *Initialization-on-demand holder* (hoặc *Double-Checked Locking*) để đảm bảo thread-safe mà không bị giảm hiệu năng. 2. Với Observer: Dùng *CopyOnWriteArrayList* để tránh lỗi *ConcurrentModificationException* khi thêm/bớt observer trong lúc thông báo."

- Kết quả đầu ra nâng cao của AI:** AI đã trả ra các block code Java tối ưu chính xác theo các từ khóa kỹ thuật được yêu cầu, minh họa trực quan cách áp dụng lớp lồng nhau (nested class) cho Singleton và cấu trúc dữ liệu Thread-safe cho Observer.



3. Đánh giá, chỉnh sửa và tích hợp

Sau hai lượt tương tác, tôi nhận thấy sản phẩm của AI rất sạch sẽ về cấu trúc nhưng mang tính rập khuôn. Quá trình tích hợp của tôi bao gồm:

- Fact-check (Kiểm chứng):** Tôi đối chiếu đoạn code áp dụng Initialization-on-demand holder của AI với tài liệu môn Lập trình hướng đối tượng của UET để đảm bảo cơ chế nạp lớp (Class Loading) của Java Virtual Machine (JVM) tự động đảm bảo tính Thread-safe mà không cần dùng từ khóa `synchronized` gây giảm hiệu năng.
- Biên tập và Tinh lọc:** Tôi loại bỏ các câu từ dẫn dắt dài dòng mang tính văn nói của AI, giữ lại các thuật ngữ chuyên ngành cốt lõi, sắp xếp lại bố cục bài tổng hợp tài liệu sao cho súc tích và đồng nhất với ngôn ngữ báo cáo kỹ thuật.

4. Trích dẫn việc sử dụng AI minh bạch

Thông báo trích dẫn: Phần nội dung so sánh ưu/nhược điểm cấu trúc và các đoạn mã nguồn minh họa cho Singleton/Observer Pattern trong báo cáo kỹ thuật đính kèm có sự hỗ trợ tra cứu, định hình cấu trúc và tối ưu hóa mã nguồn bởi mô hình ngôn ngữ lớn **Gemini** vào ngày **16/05/2026**.

III. PHÂN TÍCH CÁC VẤN ĐỀ ĐẠO ĐỨC LIÊN QUAN ĐẾN VIỆC SỬ DỤNG AI TRONG HỌC THUẬT

1. Ranh giới giữa hỗ trợ hợp lý và gian lận học thuật

Ranh giới này nằm ở **sự chủ động trong tư duy** của người học.

- Hỗ trợ hợp lý:** Là khi sinh viên đã có kiến thức nền tảng, sử dụng AI như một người bạn đồng hành để phản biện, giải thích các khái niệm khó (ví dụ: tại sao lại xảy ra

ConcurrentModificationException trong đa luồng), hoặc định hình khung sườn cho bài viết.

- **Gian lận học thuật:** Là khi sinh viên coi AI là một công cụ "làm thuê hoàn toàn". Việc ném thẳng đề bài bài tập lớn vào AI, yêu cầu viết toàn bộ mã nguồn hệ thống, sau đó copy nguyên văn nộp cho giảng viên mà bản thân không hiểu một dòng code nào hoạt động ra sao chính là hành vi gian lận, dối trá trong học tập.

2. Vấn đề quyền sở hữu trí tuệ và trích dẫn

Các mô hình AI được huấn luyện trên hàng tỷ dữ liệu mã nguồn từ các kho lưu trữ mở như GitHub hay StackOverflow. Do đó, mã nguồn hay văn bản AI trả ra không hoàn toàn là "nguyên bản" (original). Sinh viên nếu thản nhiên nhận các đoạn mã tối ưu đó là 100% do mình tự viết ra là vi phạm quyền sở hữu trí tuệ vô hình của cộng đồng mã nguồn mở. Việc trích dẫn minh bạch việc dùng AI không làm giảm đi giá trị bài làm của sinh viên, trái lại nó chứng minh sinh viên có kỹ năng nghiên cứu hiện đại và trung thực trong khoa học.

3. Tác động đến quá trình học tập và phát triển kỹ năng

- **Về mặt tích cực:** AI giải phóng sinh viên khỏi các công việc lặp đi lặp lại mang tính thủ công (như gõ các đoạn code boilerplate), giúp đẩy nhanh tốc độ nghiên cứu và tăng khả năng tự học vượt bậc.
- **Về mặt tiêu cực:** Nếu sa đà vào việc ỷ lại vào AI, sinh viên sẽ bị mất đi kỹ năng cực kỳ quan trọng của một kỹ sư IT: tư duy logic độc lập và khả năng tự xử lý lỗi hệ thống. Khi không có kết nối internet hoặc không có AI hỗ trợ, sinh viên lạm dụng công nghệ sẽ hoàn toàn bị tê liệt trước các bài toán thực tế.

IV. BỘ NGUYÊN TẮC CÁ NHÂN VỀ CÁCH SỬ DỤNG AI CÓ TRÁCH NHIỆM

Để đảm bảo bản thân luôn phát triển năng lực một cách thực chất, tôi tự thiết lập **6 nguyên tắc cốt lõi** sau khi làm việc với AI:

- **Nguyên tắc 1: Trợ lý chứ không phải Tác giả (The Assistant, Not the Author)** — Chỉ sử dụng AI để khơi gợi ý tưởng, tóm tắt và định hình cấu trúc. Bản thân luôn là người chịu trách nhiệm viết nội dung và phát triển logic cuối cùng.
- **Nguyên tắc 2: Kiểm chứng độc lập (Fact-Check Everything)** — Tuyệt đối không tin tưởng AI hoàn toàn. Mọi thuật toán, đoạn code hay kiến thức do AI cung cấp đều phải được đối chiếu lại với giáo trình, tài liệu chính thống trước khi áp dụng.
- **Nguyên tắc 3: Hiểu sâu trước khi Áp dụng (Understand Before Deploying)** — Không bao giờ copy bất kỳ dòng code nào của AI vào project cá nhân nếu bản thân không giải thích được tường tận cơ chế hoạt động của từng dòng lệnh đó.

- **Nguyên tắc 4: Minh bạch và Trung thực (Transparency & Honesty)** — Luôn công khai chủ động về việc sử dụng AI trong các bài báo cáo, bài tập lớn thông qua các danh mục trích dẫn cụ thể.
- **Nguyên tắc 5: Bảo mật và An toàn thông tin (Data Privacy)** — Tuyệt đối không tải lên các công cụ AI công cộng các dữ liệu nhạy cảm, thông tin cá nhân hoặc các mã nguồn cốt lõi thuộc dự án nghiên cứu chung của nhà trường.
- **Nguyên tắc 6: Giữ vững tư duy phản biện (Critical Thinking First)** — Luôn tương tác với AI bằng tinh thần hoài nghi khoa học, liên tục đặt câu hỏi ngược lại cho AI để tìm ra những điểm hạn chế, tối ưu của máy móc.